

Notepad++

Notepad++ supports Regex for search and replace, and can be quite useful when processing text data

Compiling Regex

In some Regex implementation such as the one in JavaScript, Regex can be compiled for better performance.

Developer corner

- `\d+` – As mentioned before, this matches one or more digit character in the pattern.

Let's say we have a text document, and we want to match the phone numbers mentioned in that document. Here we have a specific pattern with a specific number of digits. We wish to match Indian mobile numbers that take to pattern `+91xxxxxxxxx` where each `x` stands for a digit. Here is a pattern match:

```
\+91\d{10}
```

Again, we had to use `\+` instead of `+` since the latter already has a special meaning in Regex. Curly braces can be used to specify the number of times the previous expression can be repeated. Here we specify 10, but we're also free to specify a range as `{3,7}` which would match if the pattern is repeated between 3 and 7 times. Leaving it as `{3,}` would enable matching three or more times.

Finally, we have the `?` character that can be used to specify that the preceding character may or may not be present. We could, for example, use this to match both the American and British spellings of the word "colour" as such

```
colou?r
```

This will match both "color" (American spelling), and "colour" (British spelling), since the `?` specifies that the "u" is optional.

Groups

To apply `*`, `+`, `?` etc. to a group of multiple characters, we can use regular brackets to group them together. To match both "child" and "children" we could use `Child(ren)?`

Brackets also let you capture parts of an expression. Let's go back to the example we began with: URLs. We now know how to match the input URL:

```
http://www.example.com/\d{2}/\d{2}/\d{4}/articleName.html
```

The day and month are two-digit numbers, while the year is a four-digit number. But now what? We have matched the URLs we wanted to replace, but that's not enough; we need the day, month and the year matched separately so we can create a new URL. For this, we can put each of the day, month and year in separate brackets, as this will instruct the Regex engine to

Table: Regex reference

Character	Function
.	Match all characters except newline
[abc]	Match all characters inside brackets
[^abc]	Match all characters except those inside the brackets
[n-q]	Match all characters in range
\d	Match all digits
\D	Match all nondigits
\n	Match newline character
\s	Match all whitespace characters
\S	Match all non-whitespace characters
\t	Match tab character
\w	Match any word character [A-Za-z0-9_]
\W	Match any non-word character
\b	Match word boundary i.e. match between word ending and space
\B	Match non-word boundary i.e. match inside word
\	For use with special characters \\, \+, \[, etc.
*	Match preceding expression zero or more times
+	Match preceding expression one or more times
?	Preceding expression is optional, i.e. match zero or one time
[n]	Match preceding expression n times
[n,]	Match preceding expression n or more times
[n,m]	Match preceding expression between n and m times
()	Substring or capturing group

record each sub-match separately.

```
http://www.example.com/(\d{2})/(\d{2})/(\d{4})/articleName.html
```

Now when the Regex engine matches the whole text, it will also store the three subsets of the match separately so that we can use them later on. Each language has its own way of dealing with this, and often there is a simple way to replace all matches of a regular expression with text containing captured bits from the original string.

In this case, to perform the operation we desire, we would have to use the JavaScript replace function, and replace occurrences of the above Regex pattern with:

```
http://www.example.com/$3/$2/$1/articleName.html
```

Here the Regex engine automatically substitutes the `$1` for the first captured group, `$2` for the second captured group and so on.

If you find yourself using Notepad++ to play around with Regex, the above example will not work,

since Notepad++ uses a dif-

ferent syntax. In Notepad++ you will need to use `"1"` for the first

captured group, `"2"` for the second and so on. Consult the documentation of whatever software you are using to find out the syntax it uses.

Conclusion

As you may have come to realize, Regular Expressions are a very powerful tool indeed, and the best thing is that they go beyond just one programming language or tool. You'll find Regex-enabled applications everywhere. File renaming tools might use Regex to allow complex renaming operations, most IDEs support it, search tools as well. Once you master regular expressions, you'll find tons of applications.

What we've covered here are some of the basic principles of regular expressions. Many advanced features still remain out there for you to learn. Regex is an intricate tool, and many of its intricacies only become clearer as one begins using it. ■